Title Page

Title: ECL: A Deterministic Cross-Runtime Execution IR with Replayable Semantics and Dependency Interface

Author: Bin Zhang

Affiliation: independent researcher

Corresponding author: Bin Zhang

ORCID: not provided

Repository: https://github.com/joy7758/ecl-execution-compact-layer

Version: v0.1

Target journal: The Journal of Systems and Software

Article type: Research article / software systems paper

Highlights

- Deterministic execution IR maps OpenAI-style and LangChain-style traces.
- Replay emits hash-stable trace, evidence, and result artifacts.
- Loss-aware mapping records incomplete preservation instead of hiding it.
- SDK and dependency APIs expose ECL without modifying host runtimes.

Abstract

Agent runtimes increasingly expose traces, tool calls, callbacks, and event streams, but these records are usually tied to a specific framework. This paper presents Execution Compact Layer (ECL), a deterministic execution intermediate representation for agent systems. ECL maps runtime traces into four replayable surfaces: state, intent, action, and evidence. The v0.1 system includes a frozen JSON schema, OpenAI Agents SDK-style and LangChain-style trace adapters, deterministic validation and replay, an embeddable SDK, a dependency-mode API, and an MCP-shaped local wrapper. Local evaluation shows stable replay hashes, cross-runtime fixture conversion, schema validation, negative validation behavior, and field-level mapping coverage over a synthetic trace corpus. ECL is not a framework, public standard, benchmark, or external adoption claim. It is a local, reproducible software artifact for representing agent execution semantics across runtime boundaries.

Keywords

agent systems; execution traces; intermediate representation; deterministic replay; software reproducibility; trace normalization

1. Introduction

Modern agent systems produce execution traces through framework-specific surfaces: model inputs, tool calls, callback events, run trees, spans, and evidence logs. These records are useful inside their host frameworks, but they are difficult to compare, replay, or cite across runtimes without rewriting the host runtime or assuming a common execution model.

ECL addresses this gap by defining a minimal deterministic representation for agent execution. The goal is not to replace agent frameworks. The goal is to provide a compact execution IR that can be generated from heterogeneous traces and replayed into stable local artifacts.

This paper makes four contributions:

1. A four-surface execution model: state, intent, action, and evidence.
2. A deterministic hash and replay contract for local execution records.
3. A cross-runtime mapping surface for OpenAI Agents SDK-style and LangChain-style traces.
4. A dependency interface and MCP-shaped local wrapper that expose ECL without modifying host runtimes.

2. Problem Statement

Agent execution records are fragmented across runtimes. A tool call in one framework may appear as a span, callback, run node, trace event, or tool invocation object in another. Existing traces can preserve rich local detail, but they do not necessarily provide a minimal cross-runtime representation with deterministic replay semantics.

The problem is:

Given a runtime trace T from framework R,
produce a deterministic execution representation E
such that E can be validated, replayed, hashed, and cited
without modifying R or asserting full-fidelity trace preservation.

The required properties are minimality, determinism, replayability, loss awareness, and boundary preservation. ECL keeps only the execution surfaces needed for state, intent, action, and evidence. Repeated conversion and replay over unchanged inputs produces identical hashes. Adapter mappings can record incomplete preservation instead of silently claiming full fidelity. ECL references host runtime data but does not redefine the host runtime.

3. Related Work

OpenAI Agents SDK exposes tracing for agent workflows, and LangChain exposes tracing and callback-style observability for chains, tools, and model calls. These systems are runtime-specific trace surfaces. ECL differs by mapping traces into a minimal replayable IR rather than serving as the primary runtime trace system.

The Model Context Protocol defines a protocol surface for tools and context exchange. ECL's MCP-shaped local wrapper is not a conformant MCP implementation, JSON-RPC transport, published MCP server, or registry integration. It is a local anchor surface that demonstrates how an execution representation can be exposed through tool-shaped functions.

OpenTelemetry defines trace data models for distributed systems. ECL is related in spirit because it treats execution as reviewable trace data, but ECL is narrower: it targets deterministic agent execution representation, replay artifacts, and evidence bundles.

Compiler and runtime ecosystems use intermediate representations such as LLVM IR and WebAssembly to separate source-level systems from execution targets. ECL uses the same separation principle at a smaller scale: host runtimes emit or are mapped into a compact execution representation.

4. ECL Model

An ECL record is a tuple:

$$E = (S, I, A, V)$$

where:

- S is state: lifecycle, actor, persona, runtime, and correlation references.
- I is intent: operation, constraints, expected result, and evidence requirements.
- A is action: tool or operation step, execution mode, side-effect class, and parameters hash.

- V is evidence: result summary, trace references, event chain, policy decisions, and hashes.

The machine-readable binding is schemas/ecl-execution-compact-layer.schema.json. The local semantic source is SPEC.md.

5. Formal Execution Contract

Let T_r be a source trace from runtime r, M_r be a deterministic adapter for runtime r, E be an ECL object, L be a loss report, P be the schema and hash validator, R be the replay function, and H be SHA-256 over deterministic sorted-key compact JSON.

The adapter contract is:

M_r(T_r) -> (E, L)

The validation contract is:

P(E, schema) -> valid | invalid

The replay contract is:

R(E) -> (execution_trace.json, evidence_bundle.json, replay_result.json)

The determinism invariant is:

If T_r, M_r, schema, checkpoint_ref, and generated_at are unchanged,
then H(E) and H(R(E)) remain unchanged.

ECL records execution representation semantics. It does not prove that an external side effect occurred, and it does not make host runtime traces authoritative outside their source systems.

6. System Design

The schema layer defines the ECL object model and validates required fields. It is implemented in schemas/ecl-execution-compact-layer.schema.json and ecl/validator.py.

The adapter layer maps runtime traces into ECL. The v0.1 adapter surface covers OpenAI Agents SDK-style and LangChain-style traces through ecl/adapters/ and external/adoption/.

The mapping rule is:

Runtime source	ECL surface
runtime metadata, actor, correlation	state
model input, reasoning, requested operation	intent
tool call or run step	action
trace events, outputs, result refs	evidence

The replay layer validates an ECL record and emits deterministic artifacts through demo/replay_demo.py and external/adoption/replay_adapter.py.

The SDK exposes a minimal embedding surface through sdk/ecl.py and sdk/ecl_dependency.py:

```
import ecl_dependency as ecl

ecl_object = ecl.wrap(trace)
payload = ecl.emit(ecl_object)
result = ecl.verify(ecl_object)
```

The MCP-shaped local wrapper exposes the dependency API through tool-shaped functions in mcp/ecl_tool_spec.json and mcp/ecl_server_stub.py. This is a local wrapper only. It is not a conformant MCP implementation, JSON-RPC transport, registry plugin, published MCP server, or external adoption signal.

7. Architecture

```
OpenAI-style trace ----\
                        -> ECL mapping -> ECL object -> Validator -> Replay artifacts
LangChain-style trace -/                    |
                                  -> SDK dependency API
                                  -> MCP-shaped local wrapper
```

8. Evaluation

8.1 Method

The evaluation is local and fixture-based. It does not use external APIs and does not claim third-party validation. The test suite checks schema validation, OpenAI-style fixture mapping, LangChain-style fixture mapping, replay determinism, SDK API stability, dependency API stability, MCP-shaped local wrapper safety, citation reproducibility demo stability, no drift in locked core files, synthetic cross-

runtime trace-corpus conversion, loss reporting, replay stability, and field-level mapping coverage.

8.2 Current Local Results

The latest local validation run reported:

```
python3 -m unittest discover -s tests
Ran 78 tests
OK
```

The synthetic trace-corpus evaluation reported:

```
python3 experiments/evaluate_trace_corpus.py
case_count=12
by_runtime={"langchain": 6, "openai": 6}
valid_count=12
deterministic_count=12
loss_expectation_met_count=12
loss_detected_count=3
surface_coverage={"action": 12, "evidence": 12, "intent": 12, "state": 12}
loss_type_counts={"semantic": 9, "structural": 3}
evaluation_hash=sha256:2434da21056811e7eacf1ffac6944d5420ddeb2aa783f74423326a2
b54a33974
```

The negative validation-matrix evaluation reported:

```
python3 experiments/evaluate_validation_matrix.py
baseline_valid=true
case_count=8
expected_invalid_count=8
invalid_detected_count=8
expectation_met_count=8
evaluation_hash=sha256:00fe0ee8952988f7460280b40554889a890c93f9d181a6c9d8ee8b0
194b031c0
```

The field-level mapping coverage evaluation reported:

```
python3 experiments/evaluate_mapping_coverage.py
case_count=12
total_source_fields=81
direct_mapped_field_count=80
source_hash_only_field_count=1
loss_missing_field_count=4
```

evaluation_hash=sha256:8a8b820ecbdd1b4e88a2d8e07b05be2d479add0f0c6c3265292cc5a86763e43a

The citation reproducibility demo reported:

python3 examples/citation_repro_demo.py
all_valid=true
all_deterministic=true
result_hash=sha256:8684fa8963ff9ad55c36eed6b89eb15992d1b05566aa1c79ced754fac67e1cdd

The MCP-shaped local wrapper self-check reported:

verification_hash=sha256:bdcf2ddb30a0e5975782f77fa86415226c46652bbd5490a667a2f2106dd65a5f

8.3 Interpretation

The results support a narrow claim: ECL v0.1 is locally reproducible over the included fixtures, synthetic trace corpus, and deterministic entrypoints. The results do not support claims of ecosystem adoption, production reliability, third-party validation, or benchmark superiority.

9. Threats to Validity

Internal validity is limited by the synthetic and fixture-based evaluation corpus. The experiments verify local determinism, schema rejection behavior, and mapping coverage, but they do not prove production reliability.

External validity is limited by the current adapter set. The v0.1 system targets OpenAI-style and LangChain-style traces only. Other agent runtimes may require additional adapter contracts and loss models.

Construct validity is limited by the definition of replay. ECL replay verifies representational stability and artifact hash stability. It does not verify that an external side effect occurred.

Conclusion validity is limited to the locked v0.1 artifact and local test environment. The claims should not be generalized to ecosystem adoption, standardization, or benchmark superiority.

10. Discussion

ECL is most useful when execution records must be compared, replayed, or cited across runtime boundaries. The main design choice is to preserve a small

execution surface rather than reproduce every runtime-specific field. This makes the representation compact but requires explicit loss reports when mappings are incomplete.

ECL separates representation from execution. A host runtime executes. ECL records and replays the execution representation. This separation keeps the dependency interface non-invasive and avoids modifying host frameworks.

11. Limitations

- The evaluation uses local fixtures, not production traces.
- The system does not prove full-fidelity trace preservation.
- The current adapters target OpenAI-style and LangChain-style traces only.
- The MCP surface is a local stub, not a published MCP server.
- There is no external adoption signal in this version.
- There is no benchmark suite or leaderboard result.

12. Conclusion

ECL v0.1 demonstrates that agent execution traces can be mapped into a deterministic, replayable, hash-stable execution IR with a minimal dependency interface. The system is locally validated and citation-ready, but the paper-level claim remains deliberately narrow: ECL is a reproducible execution representation stack, not a framework, formal standard, production deployment, third-party validation result, or external adoption result.

CRediT Author Statement

Bin Zhang: Conceptualization, Methodology, Software, Validation, Investigation, Writing - Original Draft, Writing - Review and Editing, Project administration.

Declaration of Generative AI and AI-Assisted Technologies

AI-assisted tools, including Codex, were used for drafting assistance, repository inspection, local automation, and packaging support. The author reviewed, edited, and takes responsibility for the final content.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data Availability

All data used in the evaluation are local fixtures, synthetic trace cases, and generated evidence artifacts included in the repository. No external production trace data are used.

References

OpenAI Agents SDK tracing documentation. https://openai.github.io/openai-agents-python/tracing/

LangChain tracing documentation. https://docs.langchain.com/langsmith/tracing

Model Context Protocol specification. https://modelcontextprotocol.io/specification/

OpenTelemetry trace specification. https://opentelemetry.io/docs/specs/otel/trace/

RFC 8785 JSON Canonicalization Scheme. https://www.rfc-editor.org/rfc/rfc8785

LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html

WebAssembly Core Specification. https://webassembly.github.io/spec/core/